School of Computer Science and Artificial Intelligence

Lab Assignment # 16

Course Title : **AI Assistant Coding**
Name of Student : **B.Sai Varsha**
Enrollment No. : **2303A54065**
Batch No. : **48**

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- **Tables: Doctors, Patients, Appointments.**
- **Use AI to define constraints (unique IDs, valid dates).**
- **Generate queries:**
- **List all appointments for a specific doctor.**
- **Retrieve patient history by patient ID.**
- **Count total patients treated by each doctor.**

Expected Output:

- **Normalized schema and SQL queries with joins.**

studentdata.sql

C: > Users > rgsga > Downloads > studentdata.sql > ...

```sql
CREATE TABLE Doctors (
    doctor_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100),
    phone VARCHAR(15) UNIQUE,
    email VARCHAR(100) UNIQUE
);

CREATE TABLE Patients (
    patient_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    dob DATE NOT NULL,
    gender VARCHAR(10),
    phone VARCHAR(15) UNIQUE,
    email VARCHAR(100) UNIQUE
);

CREATE TABLE Appointments (
    appointment_id SERIAL PRIMARY KEY,
    doctor_id INT NOT NULL,
    patient_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    diagnosis TEXT,
    treatment TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id) ON DELETE CASCADE,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id) ON DELETE CASCADE,
    CONSTRAINT valid_date CHECK (appointment_date >= '2000-01-01')
);

SELECT a.appointment_id, a.appointment_date, p.name AS patient_name, a.diagnosis, a.treatment
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 101;  -- Replace 101 with the doctor's ID


SELECT a.appointment_id, a.appointment_date, d.name AS doctor_name, a.diagnosis, a.treatment
FROM Appointments a
```

SQL ▾ 1 / 1 1 - 3 of 3

doctor_id	name	specialization	phone	email
NULL	Dr. Arjun Rao	Cardiology	9876543210	arjun@hospital.com
NULL	Dr. Meera Joshi	Neurology	9876543211	meera@hospital.com
NULL	Dr. Sunil Patel	Orthopedics	9876543212	sunil@hospital.com

SQL ▾ 1 / 1 1 - 3 of 3

patient_id	name	dob	gender	phone	email
NULL	Ravi Kumar	1985-06-12	Male	9123456780	ravi@patient.com
NULL	Sita Devi	1990-02-20	Female	9123456781	sita@patient.com
NULL	Anil Sharma	1978-11-05	Male	9123456782	anil@patient.com

SQL ▾ 1 / 1 1 - 3 of 3

appointment_id	doctor_id	patient_id	appointment_date	diagnosis	treatment
NULL	1	1	2026-03-20	Flu	Rest + Medication
NULL	1	2	2026-03-22	Diabetes Check	Insulin Adjustment
NULL	2	3	2026-03-10	Migraine	Pain Relief Meds

Task 2 – Real-Time Application: Online Shopping Database
Scenario: Design a database for an E-commerce platform.
Requirements:

- **Tables: Users, Products, Orders, OrderDetails.**
- **Generate queries to:**
- **Retrieve all orders by a user.**
- **Find the most purchased product.**
- **Calculate total revenue in a given month.**
- **AI should also suggest normalization improvements and query optimization.**

Expected Output:

- **Complete schema with relationships + SQL queries for analytics.**

C: > Users > rgsga > Downloads > realtime.sql > ...

```sql
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Email TEXT UNIQUE NOT NULL,
    PasswordHash TEXT NOT NULL,
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Description TEXT,
    Price REAL NOT NULL,
    Stock INTEGER NOT NULL,
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;

CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER NOT NULL,
    OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Status TEXT DEFAULT 'Pending',
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);



CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY AUTOINCREMENT,
    OrderID INTEGER NOT NULL,
```

SQL ▾ 1 / 1 1 - 3 of 3

UserID	Name	Email	PasswordHash	CreatedAt
1	Alice Johnson	alice@example.com	hash123	2026-03-25 08:58:22
2	Bob Smith	bob@example.com	hash456	2026-03-25 08:58:22
3	Charlie Brown	charlie@example.com	hash789	2026-03-25 08:58:22

SQL ▾ 1 / 1 1 - 4 of 4

ProductID	Name	Description	Price	Stock	CreatedAt
1	Laptop	15-inch, 16GB RAM, 512GB SSD	1200.0	10	2026-03-25 08:
2	Smartphone	6.5-inch display, 128GB storage	800.0	25	2026-03-25 08:
3	Headphones	Noise-cancelling wireless headphones	150.0	50	2026-03-25 08:
4	Smartwatch	Fitness tracking smartwatch	200.0	30	2026-03-25 08:

SQL ▾ 1 / 1 1 - 4 of 4

OrderID	UserID	OrderDate	Status
1	1	2026-03-01 10:15:00	Completed
2	2	2026-03-05 14:30:00	Completed
3	1	2026-03-10 09:45:00	Pending
4	3	2026-03-15 16:20:00	Completed

SQL ▾ 1 / 1 1 - 6 of 6

OrderDetailID	OrderID	ProductID	Quantity	Price
1	1	1	1	1200.0

studentdata.sql × realtime.sql ×

C: > Users > rgsga > Downloads > realtime.sql > ...

```sql
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE o.UserID = 101;  -- Example UserID



SELECT p.ProductID, p.Name, SUM(od.Quantity) AS TotalPurchased
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID, p.Name
ORDER BY TotalPurchased DESC
LIMIT 1;


SELECT strftime('%Y-%m', o.OrderDate) AS Month,
       SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE strftime('%Y-%m', o.OrderDate) = '2026-03'
GROUP BY Month;



INSERT INTO Users (Name, Email, PasswordHash)
VALUES
('Alice Johnson', 'alice@example.com', 'hash123'),
('Bob Smith', 'bob@example.com', 'hash456'),
('Charlie Brown', 'charlie@example.com', 'hash789');


INSERT INTO Products (Name, Description, Price, Stock)
VALUES
('Laptop', '15-inch, 16GB RAM, 512GB SSD', 1200.00, 10),
('Smartphone', '6.5-inch display, 128GB storage', 800.00, 25),
('Headphones', 'Noise-cancelling wireless headphones', 150.00, 50),
('Smartwatch', 'Fitness tracking smartwatch', 200.00, 30);
```

SQLite ×

2	Bob Smith	bob@example.com	hash456	2026-03-25 08:58:22
3	Charlie Brown	charlie@example.com	hash789	2026-03-25 08:58:22

SQL ▾ 1 / 1 1 - 4 of 4

ProductID	Name	Description	Price	Stock	CreatedAt
1	Laptop	15-inch, 16GB RAM, 512GB SSD	1200.0	10	2026-03-25 08:
2	Smartphone	6.5-inch display, 128GB storage	800.0	25	2026-03-25 08:
3	Headphones	Noise-cancelling wireless headphones	150.0	50	2026-03-25 08:
4	Smartwatch	Fitness tracking smartwatch	200.0	30	2026-03-25 08:

SQL ▾ 1 / 1 1 - 4 of 4

OrderID	UserID	OrderDate	Status
1	1	2026-03-01 10:15:00	Completed
2	2	2026-03-05 14:30:00	Completed
3	1	2026-03-10 09:45:00	Pending
4	3	2026-03-15 16:20:00	Completed

SQL ▾ 1 / 1 1 - 6 of 6

OrderDetailID	OrderID	ProductID	Quantity	Price
1	1	1	1	1200.0
2	1	3	2	150.0
3	2	2	1	800.0
4	2	4	1	200.0
5	3	3	1	150.0
6	4	2	2	800.0

Task 3 – Library Database

Task: Design schema for a Library Management System.

Instructions:

- **Tables: Books, Members, Loans.**
- **Use AI to suggest indexing strategy for faster queries.**
- **Generate queries:**
- **Retrieve all books currently issued.**
- **Find overdue books (loan date > 30 days).**
- **Count number of books loaned by each member.**

Expected Output:

- **Schema + SQL queries demonstrating joins and conditions.**

studentdata.sql × realtime.sql Library.sql ×

C: > Users > rgsga > Downloads > Library.sql > SELECT * FROM Books;

```sql
CREATE TABLE Books (
    BookID INT PRIMARY KEY,
    Title VARCHAR(255) NOT NULL,
    Author VARCHAR(255),
    ISBN VARCHAR(20) UNIQUE,
    PublishedYear INT,
    Available BOOLEAN DEFAULT TRUE
);

CREATE TABLE Members (
    MemberID INT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL,
    Email VARCHAR(255) UNIQUE,
    Phone VARCHAR(15),
    MembershipDate DATE
);


CREATE TABLE Loans (
    LoanID INT PRIMARY KEY,
    BookID INT REFERENCES Books(BookID),
    MemberID INT REFERENCES Members(MemberID),
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    DueDate DATE NOT NULL
);

CREATE INDEX idx_books_available ON Books(Available);
CREATE INDEX idx_loans_book_member ON Loans(BookID, MemberID);
CREATE INDEX idx_loans_due ON Loans(DueDate);

SELECT b.BookID, b.Title, m.Name AS Borrower, l.LoanDate
FROM Books b
JOIN Loans l ON b.BookID = l.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE b.Available = FALSE AND l.ReturnDate IS NULL;
```

SQLite

SQL ▾ 1 / 1 1 - 5 of 5

BookID	Title	Author	ISBN	PublishedYear	Available
1	The Great Gatsby	F. Scott Fitzgerald	9780743273565	1925	1
2	1984	George Orwell	9780451524935	1949	1
3	To Kill a Mockingbird	Harper Lee	9780061120084	1960	1
4	The Catcher in the Rye	J.D. Salinger	9780316769488	1951	1
5	Pride and Prejudice	Jane Austen	9780141439518	1813	1

SQL ▾ 1 / 1 1 - 4 of 4

LoanID	BookID	MemberID	LoanDate	ReturnDate	DueDate
1001	1	101	2026-02-20	NULL	2026-03-21
1002	2	102	2026-01-15	2026-02-10	2026-02-14
1003	3	103	2026-02-01	NULL	2026-03-02
1004	4	104	2026-03-10	NULL	2026-04-09

SQL ▾ 1 / 1 1 - 4 of 4

MemberID	Name	Email	Phone	MembershipDate
101	Alice Johnson	alice@example.com	9876543210	2023-01-15
102	Bob Smith	bob@example.com	9876543211	2023-02-20
103	Charlie Brown	charlie@example.com	9876543212	2023-03-05
104	Diana Prince	diana@example.com	9876543213	2023-04-10

Task 4: Optimize Queries

Instructions:

- **Use AI tools to analyze query efficiency.**
- **Optimize inefficient queries using joins, indexes, or reducing subqueries.**
- **Test optimized queries for correctness.**

Starter Code Example:

```sql
-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Expected Output:

- **Both queries return the same result: all books written by UK authors.**
- **Optimized query performs faster for larger datasets.**

SQL ▾ 1 / 1 1 - 3 of 3

BookID	Title	Borrower	LoanDate
1	The Great Gatsby	Alice Johnson	2026-02-20
3	To Kill a Mockingbird	Charlie Brown	2026-02-01
4	The Catcher in the Rye	Diana Prince	2026-03-10

SQL ▾ 1 / 1 1 - 2 of 2

Title	Borrower	LoanDate	DueDate
To Kill a Mockingbird	Charlie Brown	2026-02-01	2026-03-02
The Great Gatsby	Alice Johnson	2026-02-20	2026-03-21

SQL ▾ 1 / 1 1 - 4 of 4

MemberID	Name	BooksLoaned
101	Alice Johnson	1
102	Bob Smith	0
103	Charlie Brown	1
104	Diana Prince	1

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.

• Define relationships:

• Each student can register for multiple courses.

• Each course is taught by a faculty member.

• Write SQL queries to:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

studentdata.sql × univwesrity.sql ×

lab 16 > univwesrity.sql > ...

```sql
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100) UNIQUE
);


CREATE TABLE Faculty (
    faculty_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department VARCHAR(100)
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    faculty_id INT,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);


CREATE TABLE Registrations (
    registration_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    registration_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);


SELECT s.student_id, s.first_name, s.last_name
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
JOIN Courses c ON r.course_id = c.course_id
```

SQLite

SQL ▾ 1 / 1 1 - 3 of 3

student_id	first_name	last_name
1	Rahul	Sharma
2	Priya	Mehta
4	Sneha	Patel

SQL ▾ 1 / 1 1 - 1 of 1

faculty_id	first_name	last_name	course_count
101	Dr. Anil	Kumar	3

SQL ▾ 1 / 1 1 - 1 of 1

course_id	course_name	total_registrations
201	Database Systems	3